DESIGN AND ANALYSIS OF ALGORITHMS

LECTURE 2

Analysis of Algorithms

Review the Previous Lesson

- Some key concepts
  - Problem; Algorithm; Data Structure; Program.
- About Complexity and Analysis of Algorithms
- Turing Machine
  - Description; Structure; and Operation
  - Formal definition ➡ Algorithm
- Primitive Recursive Function
  - Basic primitive recursive functions
  - Composition; Primitive recursion

➡ Computable functions

Outline

- The Analysis Framework
- Analyze the complexity of Algorithms
- Prove the correctness of Algorithms
- Exercises

[Anany's book Chapter 2, page 41]

The Analysis Framework

- ✓ Goals of Algorithm Analysis
- ✓ Input Data Size and Basic Operation
- ✓ Orders of Growth
- ✓ Types of analysis

Goals of Algorithm Analysis

- How can we trust the algorithm?
  - Proving the Correctness.
- How to compare the algorithms?
  - Assessing the Effectiveness.

Effectiveness: Two kinds of algorithm efficiency

- Time efficiency - how fast the algorithm runs?
  - ⇨ Time complexity – Độ phức tạp thời gian.
- Space efficiency - how much extra memory it uses?
  - ⇨ Space complexity – Độ phức tạp không gian.

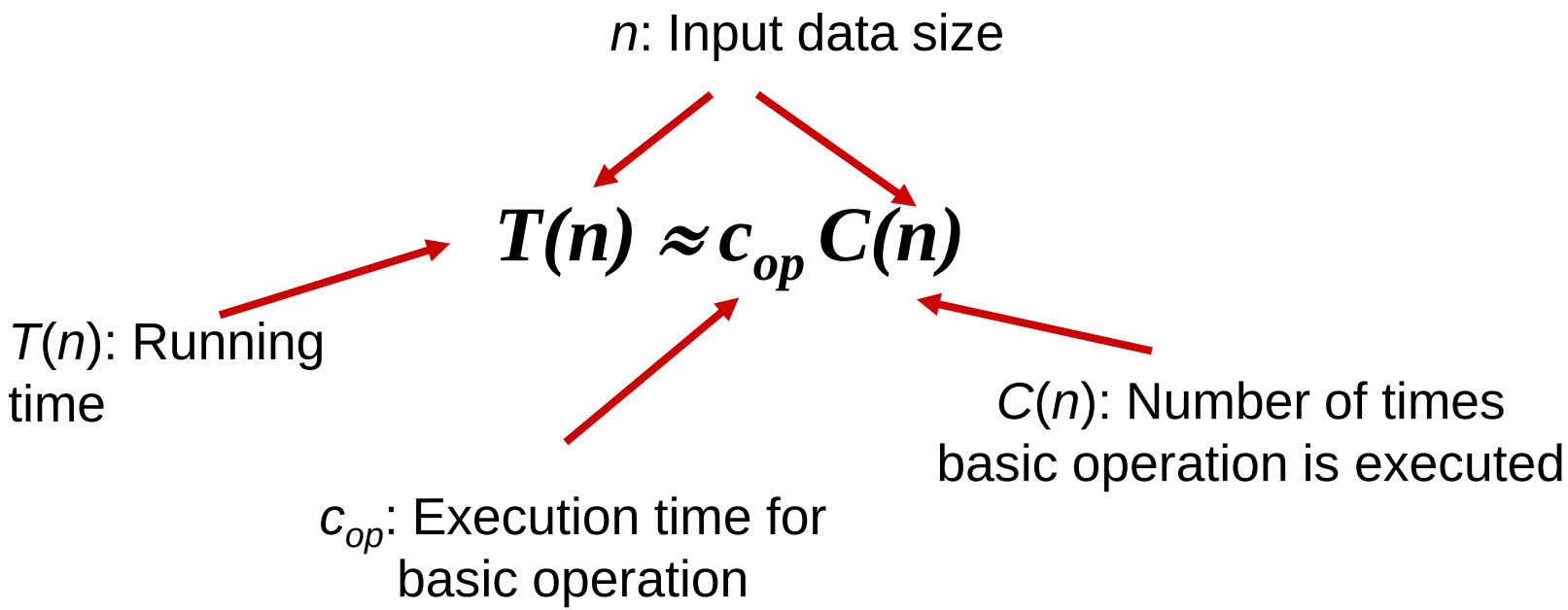
Goals of Algorithm Analysis

- In theoretical analysis of algorithms, the complexity is commonly estimated in the asymptotic sense, i.e., to estimate the complexity as a function of arbitrarily large input.

  Trong phân tích lý thuyết thuật toán, độ phức tạp thường được ước lượng theo nghĩa tiệm cận, tức là ước tính độ phức tạp như một hàm đối với dữ liệu đầu vào có kích thước lớn tùy ý.

Goals of Algorithm Analysis

- Theoretical Analysis of Time Complexity:
  - Time complexity is determined by the number of repetitions of the basic operations as a function of the input data size.



Input Data Size and Basic Operation

- Input Data Size: number of elements or value of input.
- Basic Operation: compare, arithmetic operation, element accesses...

Problem	Input size	Basic operation
Search for key in a list	Number of items in list n	Key comparison
Multiply two matrices	Dimensions of matrices	Floating-point multiplication
Compute a^n	Value of n	Floating-point multiplication
Find a path on a graph	Size of graph (Number of vertices and edges)	Visiting a vertex or traversing an edge

Goals of Algorithm Analysis

- Theoretical Analysis of Time Complexity:
  - Example with Insertion Sort

Instruction	Running Time
`InsertionSort(A, n) {`	
`  for i = 2 to n {`	$c_1 n$
`  key = A[i]`	$c_2(n\text{-}1)$
`  j = i - 1`	$c_3(n\text{-}1)$
`  while (j > 0) and (A[j] > key) {`	$c_4 T$
`  A[j+1] = A[j]`	$c_5(T\text{-}(n\text{-}1))$
`  j = j - 1`	$c_6(T\text{-}(n\text{-}1))$
`  }`	
`  A[j+1] = key`	$c_7(n\text{-}1)$
`  }`	
`}`	

$T = t_2 + t_3 + \ldots + t_n$ where t_i is the number of times the **while loop condition is tested** for value j at step i.

Goals of Algorithm Analysis

- Theoretical Analysis of Time Complexity:
  - Example with Insertion Sort

$$\rightarrow T(n) = c_1 n + c_2(n\text{-}1) + c_3(n\text{-}1) + c_4 T + c_5(T\text{-}(n\text{-}1)) + c_6(T\text{-}(n\text{-}1)) + c_7(n\text{-}1)$$
$$= c_8 T + c_9 n + c_{10}, \qquad where\ T = t_2 + t_3 + \ldots + t_n$$

Best Case: Array already sorted $\rightarrow$ Loop condition fails immediately $\rightarrow t_i = 1, \forall i$

$\rightarrow T(n) = c_8 n + c_9 n + c_{10} = an + b,$ (Linear).

Worst Case: Array reverse sorted $\rightarrow$ Loop runs fully for each $i \rightarrow t_i = i$

$$\rightarrow T(n) = c_8 \sum_{i=2}^{n} i + c_9 n + c_{10} = c_8 \frac{n(n+1)}{2} + c_9 n + c_{10} = an^2 + bn + c,$$ (Quadratic)

Average Case: Assume key is inserted in the middle $\rightarrow t_i \approx i / 2.$

$$\rightarrow T(n) \approx c_8 \sum_{i=2}^{n} \frac{i}{2} + c_9 n + c_{10} = c_8 \frac{1}{2} \frac{n(n+1)}{2} + c_9 n + c_{10} = an^2 + bn + c,$$ (Quadratic)

Orders of Growth

- The Running Time Formula $T(n) \approx c_{op}.C(n)$
- Two factors affect the growth of $T(n)$
  - c_{op}: Cost per an operation (Hardware/Language dependent)
  - $C(n)$: Number of operations (Algorithm dependent)

Better Hardware	Larger Input
Effect of doubling hardware speed ($c_{op} \rightarrow 1/2 c_{op}$)?	Impact of doubling input data size ($n \rightarrow 2n$)?
Running time decreases by **half** (*Hardware gives constant improvement*).	**Depends** on the Algorithm! - If $C(n) \sim n$ (Linear): Time doubles (2x). - If $C(n) \sim n^2$ (Quadratic): Time quadruples (4x) - If $C(n) \sim n^2$ (Cubic): Time increases (8x)

Focus on $C(n)$ – the Order of Growth – to evaluate algorithm efficiency.

Orders of Growth

- Value of several functions important for analysis of algorithms

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

[Anany's book, page 46]

Orders of Growth

- Common Time Complexity Classes

Time Complexity	Name	Description
1	Constant	Whatever is the input size n, these functions take a constant amount of time.
$logn$	Logarithmic	These are slower growing than even linear functions.
n	Linear	These functions grow linearly with the input size n.
$nlogn$	Linear Logarithmic	Faster growing than linear but slower than quadratic.
n^2	Quadratic	These functions grow faster than the linear logarithmic functions.
n^3	Cubic	Faster growing than quadratic but slower than exponential.
2^n	Exponential	Faster than all of the functions mentioned here except the factorial functions.
$n!$	Factorial	Fastest growing than all these functions mentioned here.

Orders of Growth

- The Power of Exponential Growth

Volume of a grain: $2mm^3$, Total $V=2^{65}\ mm^3 \approx$ 12.000 Giza pyramids

http://mathgardenblog.blogspot.com/2015/04/chess.html

Types of Analysis

- Best-case, Average-case, Worst-case

  For some algorithms the efficiency depends on the type of input:

  - **Worst case**: W(*n*) – maximum over inputs of size *n*
  - **Best case**: B(*n*) – minimum over inputs of size *n*
  - **Average case**: A(*n*) – "average" over inputs of size *n*
    - Number of times the basic operation is executed on a typical input
    - NOT the average of the worst and best cases
    - Average-case analysis depends on assumptions about the probability distribution of inputs.

Analyze the complexity of Algorithm

- ✓ Asymptotic Notations
- ✓ Establishing rate of growth relation

Asymptotic Notations

- Asymptotic notations are syntax for presenting the upper and lower bounds of algorithm complexity.
  - Best case but What is the lower bound?
  - Worst case but What is the upper bound?
  - Average case but What is the interval bound?
- A method to compare functions that ignores constant factors and small input sizes.
- Three main Asymptotic notations:
  - O - Big-oh
  - Ω - Big-omega
  - Θ - Big-theta

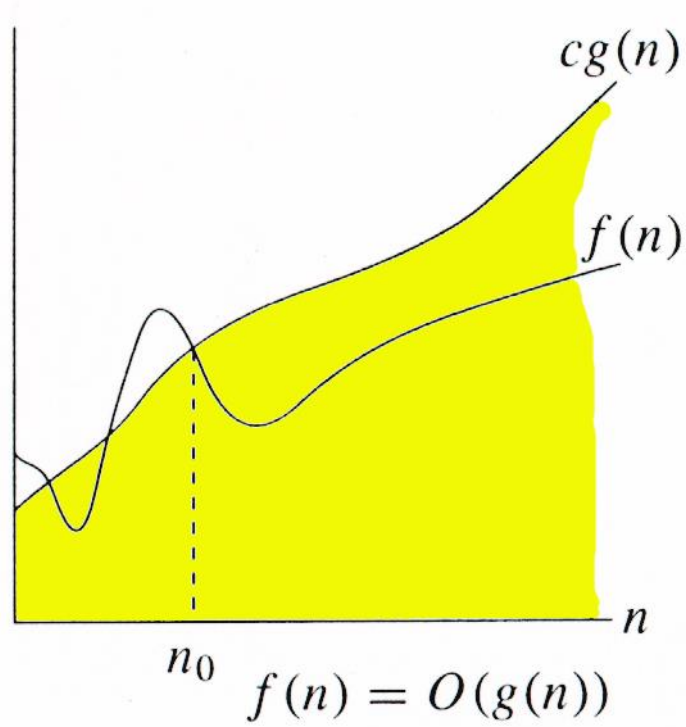
Asymptotic Notations

- O - Big-oh (đọc là O lớn – Tiệm cận trên)
  - Definition:

    $f(n) = O(g(n))$ if exist c, n_0 such

    $f(n) \leq c.g(n)$ for all $n \geq n_0$
  - Meaning:

    $f(n)$ grows at most as fast as $g(n)$



O notation: Worst-case upper bound.
Interested in the smallest valid bound.

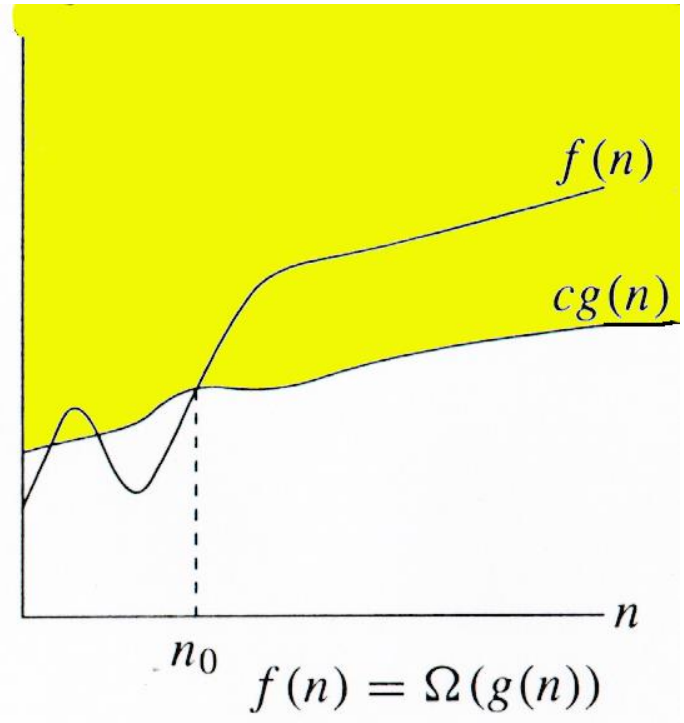
Asymptotic Notations

- Ω - Big-omega (đọc là Omega lớn – tiệm cận dưới)
  - Definition:

    $f(n) = \Omega(g(n))$ if exist c, n_0 such

    $f(n) \geq c.g(n)$ for all $n \geq n_0$
  - Meaning:

    $f(n)$ grows at least as fast as $g(n)$



Ω notation: Best-case lower bound.
Interested in the largest valid bound.

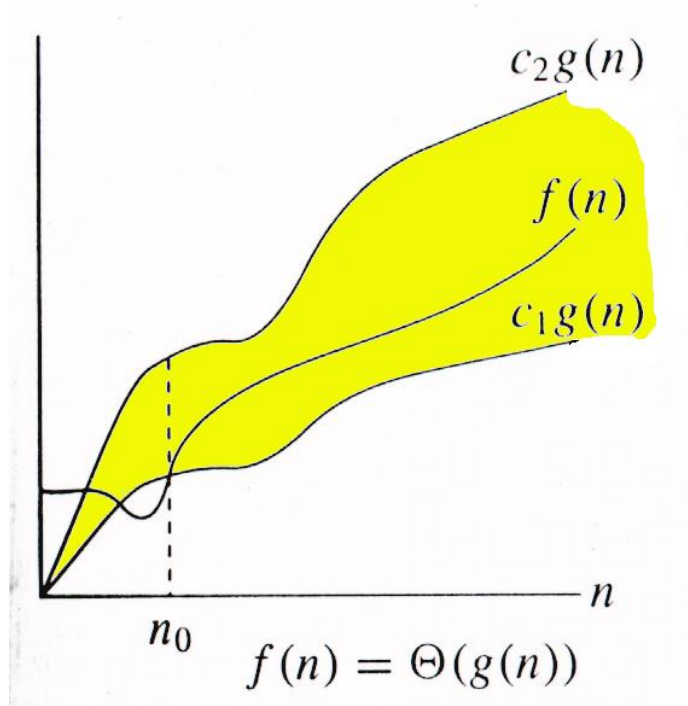
Asymptotic Notations

- Θ - Big-theta (đọc là Theta lớn - tiệm cận chặt)
  - Definition:

    $f(n) = \Theta(g(n))$ if exist c_1, c_2, n_0 such

    $c_1.g(n) \le f(n) \le c_2.g(n)$ for all $n \ge n_0$
  - Meaning:

    $f(n)$ grows at the same rate as $g(n)$



Θ notation: Average-case tight bound.
Interested in the exact order of growth.

Asymptotic Notations

- Example
  - Time complexity of *Insertion Sort* is $O(n^2)$, isn't it?

    $f(n) = an^2+bn+c = O(n^2)$
  - Prove:

    $f(n) = an^2 + bn + c$

    $\leq (a + b + c)n^2 + (a + b + c)n + (a + b + c)$

    $\leq 3(a + b + c)n^2$ with $n \geq 1$

    Choose $c' = 3(a + b + c)$, $n_0 = 1$ $\Rightarrow f(n) \leq c'.n^2$

    or $f(n) = O(n^2)$ ■
  - Is $f(n) = an^2+bn+c = O(n^3)$? - Yes, but need the smallest.
  - More examples in Anany's book page 53-55

Useful Property of the Asymptotic Notations

- Theorem:

If $t_1(n) = O(g_1(n))$ and $t_2(n) = O(g_2(n))$, then

$$t_1(n) + t_2(n) = O(\max\{g_1(n), g_2(n)\}).$$

  - Prove: See [Anany's book page 56].
  - Property useful in analyzing algorithms that comprise two consecutively executed parts.

Useful Property of the Asymptotic Notations

- Other properties of asymptotic relationship:

Transitivity:

$$f(n) = \Theta(g(n)) \ \& \ g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \ \& \ g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \ \& \ g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$$

Reflexivity:

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

Symmetry and Transpose Symmetry:

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$$

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$

Establishing rate of growth relation

- Establishing rate of growth relation
  - Given 2 complexity functions f(n) và g(n). Lets determine $f(n) = *(g(n))$ with * is O or Ω or Θ?
- Three methods

Using mathematical definition:

Find constants c, n_0 satisfy the conditions

Using inductive proof:

Example: $\log n = O(n)$ or $\log(n) \leq c.n$

Base: $n = 1 \Rightarrow 0 < 1$ - is true

Inductive step:

Assume $\log(n) \leq n$ when $n>1$

Then $\log(n+1) \leq \log(n+n) = \log(2n) = \log n + 1 \leq n+1$ ■

Using limits (when $n \to \infty$)

Establishing rate of growth relation

- **Using limits** (when $n \to \infty$)

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \Rightarrow f(n) = \mathrm{O}(g(n)) \\ \infty & \Rightarrow f(n) = \Omega(g(n)) \\ const & \Rightarrow f(n) = \Theta(g(n)) \\ unknow & \Rightarrow \text{ no relation} \end{cases}$$

Example:

$$\text{Given } f(n) = n\sqrt{n} \text{ and } g(n) = n^2 - n$$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n\sqrt{n}}{n^2 - n} = \lim_{n\to\infty} \frac{\sqrt{n}}{n-1} = 0$$

$$\Rightarrow f(n) = \mathrm{O}(g(n))$$

More examples [Anany's book page 57]

Establishing rate of growth relation

- Calculus techniques for computing limits
  - L'Hôpital's rule

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \lim_{n\to\infty} \frac{t'(n)}{g'(n)}$$

  - Stirling's formula

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \quad \text{for large values of } n.$$

Proving Algorithm Correctness

- ✓ Algorithm’s correctness
- ✓ Correctness of Recursive algorithm
- ✓ Correctness of Iterative algorithm

Correctness of Algorithms

- How do we know that an algorithm works?

  The answer leads to the need for checking correctness.

- Methods for establishing algorithm correctness
  - Testing: Try the algorithm on sample inputs.

    Testing may fail to reveal subtle or obscure bugs.

    Using testing alone is insufficient.
  - Correctness proof: Mathematical proof of correctness

    Correctness proofs may also contain errors.

A combination of testing and correctness proof provides greater confidence in algorithm correctness.

Correctness of Recursive algorithms

- Recursive algorithm

  A **recursive algorithm** is an algorithm that calls itself on smaller (or simpler) instances of the same problem.

- Proving correctness by induction:

  Prove by mathematical induction on the "size" of the problem:

  - ***Base case:*** The base case of the recursion is correct.
  - ***Inductive hypothesis***: Assume that the recursive call works correctly for input size *n.*
  - ***General case****:* Using the hypothesis, prove that the algorithm works correctly for size *n*+1.
  - ***Termination***:Since the recursion eventually reaches the base case, the algorithm terminates.

Correctness of Recursive algorithms

- Example
  - Recursive algorithm find maximum of a list:

    Maximum(A,n) ≡ //*Find the maximum*

    if (n==1) return (A_1)

    else return(max(Maximum(A,n-1),A_n)

    End.
  - Claim: maximum(n) returns max{A_1,A_2,…,A_n} for all $n \geq 1$
  - Proof by induction on n:
    - Base case: n = 1, maximum(A,n) returns A_1 as claimed
    - Inductive assumption: maximum(A,n) returns max {A_1,A_2,…,A_n}
    - General case: Maximum(A,n+1) returns max(Maximum(A,n),A_{n+1})

      = max(A_1,…, A_n, A_{n+1})

Correctness of Iterative algorithms

- Iterative algorithm
  - A non-recursive algorithm that uses one or more loops.
  - Prove the correctness by loop invariant
- Loop invariant
  - A logical statement about program variables that is true before and after each iteration of the loop. - *Biểu thức logic luôn đúng ở mỗi lần lặp*
  - Loop invariants are used to prove that an algorithm terminates and produces correct results – *Dùng để chứng minh thuật toán dừng và cho kết quả đúng.*
  - For algorithms with nested loops, the loop invariant is usually established starting from the innermost loop. – *Với các vòng lặp lồng nhau sẽ bắt đầu từ vòng lặp trong cùng.*

Correctness of Iterative algorithms

- Properties of loop invariant
  - **Initialization** (*khởi tạo*): The loop invariant is true before the first iteration of the loop.
  - **Maintenance** (*duy trì*): If the loop invariant is true at the beginning of an iteration, it remains true at the beginning of the next iteration.
  - **Termination** (*kết thúc*): The loop eventually terminates.

    When the loop terminates, the loop invariant, together with the termination condition, implies the correctness of the loop and the algorithm.

Correctness of Iterative algorithms

- Example: Non-recursive algorithm find maximum of a list:

  Maximum(A,n) ≡ *//Find the maximum of the list has n items*

  $m = A_1$

  for (i=2;i<=n;i++) if (m<A_i) $m = A_i$

  return (m)

  End.

  Loop invariant: $m_j = \max(A_1,\ldots,A_j)$

  - **Initialization**: $m_1 = A_1 = \max(A_1)$ - is true
  - **Maintenance**: if $m_j = \max(A_1,\ldots,A_j)$, have $m_{j+1} = \max(m_j,A_{j+1}) = \max(A_1,\ldots,A_{j+1})$
  - **Termination**: when $i=n+1$, after t iterations ($t = n+1-2+1=n$)

    $$m_t = \max(A_1,\ldots A_t) = \max(A_1,\ldots A_n)$$

  Because the loop invariant is maintained and the loop terminates, the Maximum algorithm is correct.

  See more [Rodney R. Howell, Algorithm: a Top-Down Approach, Chapter 2]

Exercises

- Measuring input size and orders of growth.
- Establishing growth-rate relationships between functions.
- Designing and analyzing a sorting algorithm.
- Further details are provided in Hw2_AlgorithmAnalysis.